

Temperature and Humidity Monitoring

Name: Brinda Jeniffer Santiner.R
Class: B.Sc(MEC)
Register Number: U03ZW24S0015

Team members: Mahalakshmi and shivaranjan

Problem Statement:

Monitoring temperature and humidity manually is difficult and does not provide continuous information. There is a need for a low-cost system that can automatically measure environmental conditions and provide real-time monitoring.

The proposed system uses ESP32 and DHT22 to collect temperature and humidity data. An LED can also be used as an indicator when a particular condition is detected.

Objectives:

The main objectives are:

To measure temperature using the DHT22 sensor.

To measure relative humidity.

To process the sensor data using the ESP32.

To provide continuous monitoring of environmental conditions.

To indicate a particular condition using an LED.

To enable future integration with an IoT cloud platform such as ThingSpeak.

To develop practical knowledge of IoT sensors, ESP32 programming, and cloud monitoring.

Hardware and Software:

Esp32

DHT22 Temperature sensor

Jumper wires

LED

Resistor

Thingspeak

Working Principle:

Step 1: Sensing

The DHT22 sensor detects the surrounding temperature and humidity.

Step 2: Data transmission

The DHT22 sends the measured values to the ESP32 through its digital data pin.

Step 3: Processing

The ESP32 reads and processes the temperature and humidity values.

Step 4: Display/monitoring

The values can be displayed through the Serial Monitor and, when Wi-Fi and ThingSpeak are configured, uploaded to the cloud for remote monitoring.

Step 5: LED indication

The ESP32 controls the LED according to the programmed condition. For example, the LED can turn ON when temperature or humidity crosses a predefined threshold.

Applications:

This system can be used in:

Smart homes – monitoring indoor environmental conditions.

Agriculture – monitoring temperature and humidity in farms or greenhouses.

Weather monitoring – collecting local environmental data.

Warehouses – monitoring storage conditions.

Smart classrooms – checking room environmental conditions.

Industrial environments – monitoring temperature and humidity.

Greenhouses – helping maintain suitable conditions for plants.

Future Scope:

The project can be improved in several ways:

Add ThingSpeak cloud monitoring for real-time graphs.

Add a mobile application for remote monitoring.

Add more sensors such as air-quality, light, or soil-moisture sensors.

Add an LCD/OLED display.

Add automatic control of a fan or exhaust system when temperature becomes high.

Store historical sensor data for analysis.

Add alerts through suitable IoT notification services.

Use multiple sensors for monitoring different locations.

Develop a complete smart environmental monitoring system.

Wokwi link:

<https://wokwi.com/projects/475867704193573889>

Thingspeak link:

https://thingspeak.mathworks.com/channels/3504258/private_show

Github link:

<https://github.com/jenifferbrinda-bop/Temperature-and-Humidity-Monitoring->

Coding:

```
import dht
import time
import network
import urequests
from machine import Pin

# -----
# DHT22 SENSOR
# -----
sensor = dht.DHT22(Pin(15))

# -----
# LED
# -----
led = Pin(2, Pin.OUT)

# -----
# WI-FI
# -----
wifi = network.WLAN(network.STA_IF)
wifi.active(True)

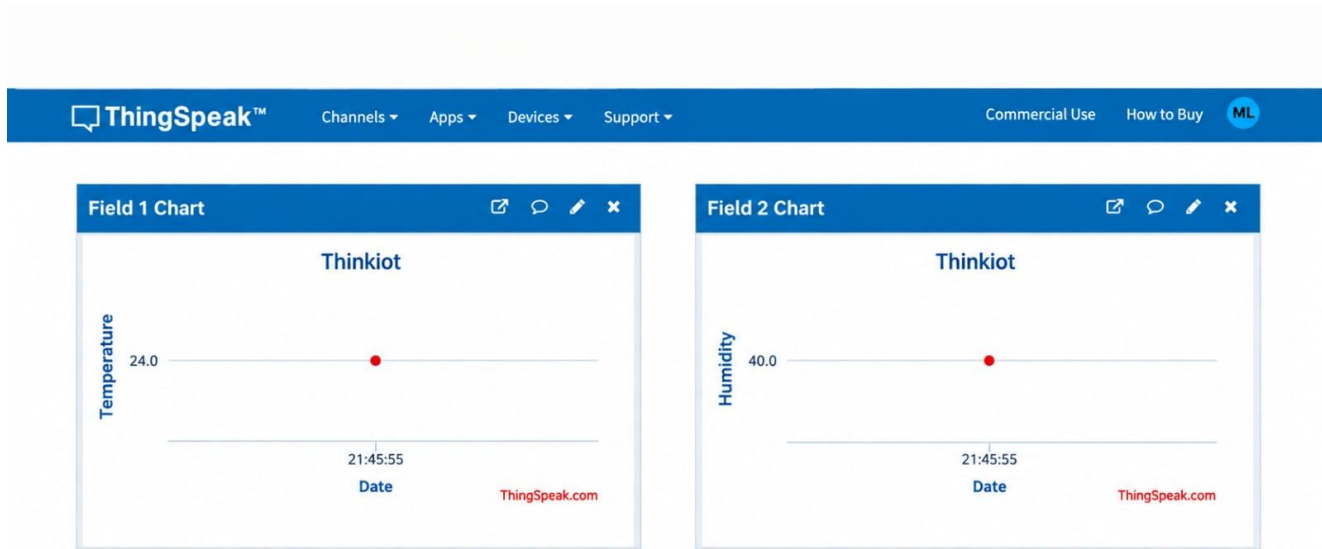
wifi.connect("Wokwi-GUEST", "")
print("Connecting to Wi-Fi...")
while not wifi.isconnected():
    time.sleep(1)

print("Wi-Fi connected!")
print(wifi.ifconfig())

# -----
# THINGSPEAK
# -----
API_KEY = "TD210S9JCKQC9788"

# -----
# MAIN PROGRAM
# -----
while True:
    try:
        # Read DHT22
        sensor.measure()
        temperature = sensor.temperature()
        humidity = sensor.humidity()
        # Display values in Serial Monitor
        print("-----")
        print("Temperature:", temperature, "C")
        print("Humidity:", humidity, "%")
        # -----
        # LED CONTROL
        # -----
        if temperature >= 30:
            led.on()
            print("LED: ON - High Temperature")
        else:
            led.off()
            print("LED: OFF - Normal Temperature")
        # -----
        # SEND DATA TO THINGSPEAK
        # -----
        url = (
            "https://api.thingspeak.com/update"
            "?api_key={}"
            "&field1={}"
            "&field2={}"
        ).format(API_KEY, temperature, humidity)
        response = urequests.get(url)
        print("ThingSpeak response:", response.text)
        response.close()
    except Exception as e:
        print("Error:", e)
    # ThingSpeak update interval
    time.sleep(15)
```

Thingspeak output:



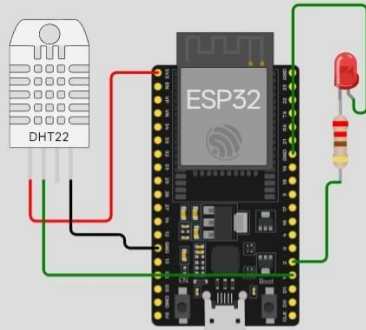
Wokwi output:

```
ets Jul 29 2019 12:21:46

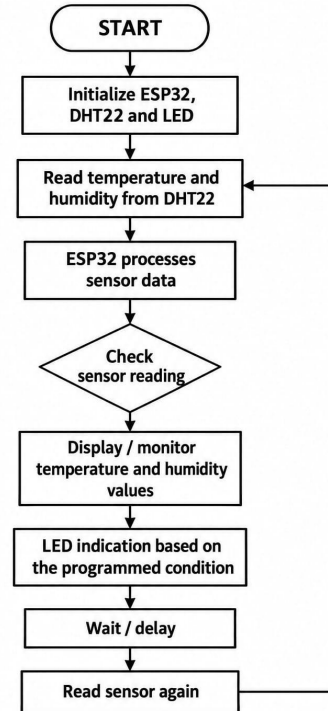
rst:0x1 (POWERON RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00

mode:DIO, clock div:2
load:0x3fff0030,len:4200
load:0x40078000,len:15236
load:0x40080400,len:3332
entry 0x400805a8
Connecting to Wi-Fi...
Wi-Fi connected!
('10.10.0.2', '255.255.0.0', '10.0.0.1', '10.0.0.1')
-----
Temperature: 24.0 C
Humidity: 40.0 %
LED: OFF - Normal Temperature
ThingSpeak response: 1
```

Circuit Connection:



WORKING FLOW – ESP32 TEMPERATURE & HUMIDITY MONITORING



Learning reflection:

Through this experiment, I learned:

How an ESP32 works as an IoT controller.

How to interface a DHT22 sensor with ESP32.

How temperature and humidity are measured digitally.

How to control an LED using a microcontroller.

How sensors and actuators communicate with a microcontroller.

The basic concept of IoT-based remote monitoring.

How Wi-Fi can be used to send sensor data to a cloud platform.

How hardware connections and software programming work together to create an IoT application.

Conclusion:

The IoT-Based Temperature and Humidity Monitoring System successfully demonstrates how an ESP32 can collect environmental information using a DHT22 sensor. The measured temperature and humidity can be processed by the ESP32 and, with IoT connectivity, uploaded to a cloud platform such as ThingSpeak for remote monitoring. The LED provides a simple visual indication based on programmed conditions.

Overall, this experiment provides a basic foundation for developing more advanced IoT-based environmental monitoring and automation systems.

